

Tugas 2: Machine Learning – Artificial Neural Network (ANN)

Muhammad Zaidan Ramdhan - 0110222040

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: muha22040ti@student.nurulfikri.ac.id

Laporan ini bertujuan untuk menerapkan Artificial Neural Network (ANN) dengan arsitektur Multi-Layer Perceptron (MLP) pada dataset citra MNIST (Handwritten Digits). Dataset MNIST terdiri dari gambar angka tulisan tangan berukuran 28×28 piksel dengan sepuluh kelas (0–9), sehingga permasalahan yang dihadapi adalah klasifikasi multi-kelas. Melalui percobaan ini, mahasiswa diharapkan dapat memahami penerapan ANN pada data citra serta mengamati performa model MLP dalam mengklasifikasikan data non-tabular.

Pada tahap preprocessing, setiap citra diubah menjadi vektor satu dimensi melalui proses flattening dan dinormalisasi ke dalam rentang 0–1 untuk meningkatkan stabilitas pelatihan model. Label kelas dikonversi ke dalam format one-hot encoding, kemudian model MLP dilatih menggunakan beberapa lapisan tersembunyi dengan fungsi aktivasi ReLU dan lapisan output Softmax. Hasil percobaan menunjukkan bahwa model mampu mencapai akurasi yang tinggi pada data pengujian, sehingga dapat disimpulkan bahwa MLP efektif digunakan untuk klasifikasi citra sederhana dan memberikan perbedaan proses yang jelas dibandingkan penerapan ANN pada dataset tabular.

1. Tugas mandiri – Latihan

```
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical
```

Gambar 1. 2

Pada *Gambar* di atas, merupakan sebuah code untuk import beberapa library yang akan kita gunakan.

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()

print("X_train:", X_train.shape)
print("y_train:", y_train.shape)
print("X_test:", X_test.shape)
print("y_test:", y_test.shape)

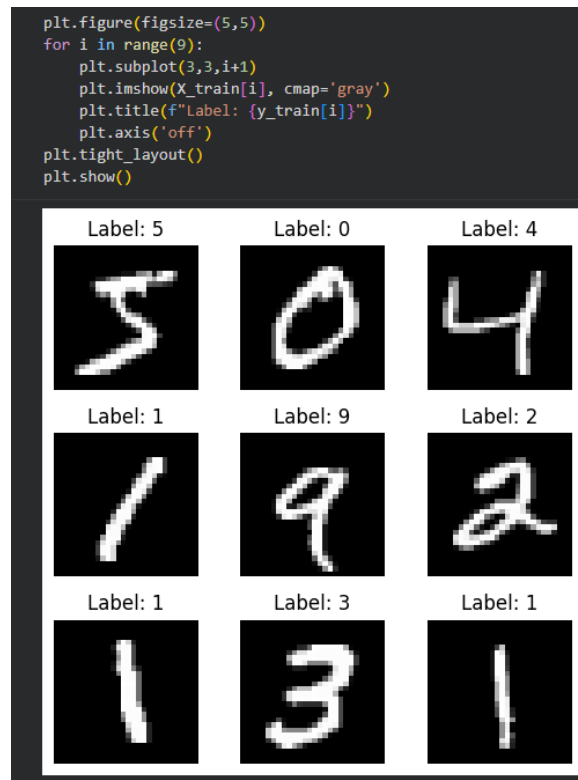
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 0s 0us/step
X_train: (60000, 28, 28)
y_train: (60000,)
X_test: (10000, 28, 28)
y_test: (10000,)
```

Gambar 1. 3

Dataset MNIST dimuat menggunakan fungsi `mnist.load_data()` yang otomatis mengunduh dataset saat pertama kali dijalankan.

Dataset terdiri dari:

- 60.000 data training
 - 10.000 data testing
- Setiap citra memiliki ukuran 28×28 piksel dalam bentuk grayscale.



Gambar 1. 4

Visualisasi dilakukan untuk memastikan bahwa data yang dimuat sesuai dengan label angka tulisan tangan (0–9).

Setiap gambar ditampilkan dalam skala abu-abu (grayscale).

```
X_train = X_train.reshape(X_train.shape[0], 28*28)
X_test = X_test.reshape(X_test.shape[0], 28*28)
```

Gambar 1. 5

Penjelasan:

Kolom `diagnosis` bertipe kategorikal sehingga perlu diubah ke bentuk numerik:

M (Malignant) → 1

B (Benign) → 0

```
X_train = X_train.reshape(X_train.shape[0], 28*28)
X_test = X_test.reshape(X_test.shape[0], 28*28)
```

Gambar 1. 6

MLP menerima input berupa vektor 1 dimensi, maka citra 28×28 diubah menjadi vektor dengan panjang 784 fitur.

```
X_train = X_train / 255.0
X_test = X_test / 255.0
```

Gambar 1.7

Nilai piksel dinormalisasi dari rentang 0–255 menjadi 0–1 untuk mempercepat konvergensi dan meningkatkan stabilitas pelatihan model.

```
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

Gambar 1.8

Label kelas (0–9) dikonversi menjadi format one-hot encoding karena klasifikasi bersifat multi-kelas.

```
model = Sequential()

model.add(Dense(128, activation='relu', input_shape=(784,)))
model.add(Dense(64, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

Gambar 1.9

Model MLP terdiri dari:

- Input layer: 784 neuron
- Hidden layer 1: 128 neuron dengan aktivasi ReLU
- Hidden layer 2: 64 neuron dengan aktivasi ReLU
- Output layer: 10 neuron dengan aktivasi Softmax

Aktivasi Softmax digunakan untuk menghasilkan probabilitas tiap kelas.

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Gambar 1.10

Penjelasan:

- Adam optimizer dipilih karena adaptif dan efisien
- Categorical Crossentropy digunakan untuk klasifikasi multi-kelas
- Accuracy digunakan sebagai metrik evaluasi

```
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=128,
    validation_split=0.2
)
```

Epoch	Progress	Time	Step	Accuracy	Loss	Val Accuracy	Val Loss
Epoch 1/10	375/375	3s	5ms/step	0.8084	0.6618	0.9498	0.1833
Epoch 2/10	375/375	2s	4ms/step	0.9548	0.1553	0.9628	0.1260
Epoch 3/10	375/375	3s	7ms/step	0.9691	0.1066	0.9671	0.1093
Epoch 4/10	375/375	2s	4ms/step	0.9779	0.0774	0.9697	0.0986
Epoch 5/10	375/375	2s	5ms/step	0.9813	0.0613	0.9695	0.1040
Epoch 6/10	375/375	2s	4ms/step	0.9873	0.0459	0.9763	0.0865
Epoch 7/10	375/375	2s	4ms/step	0.9889	0.0384	0.9750	0.0915
Epoch 8/10	375/375	2s	4ms/step	0.9913	0.0299	0.9743	0.0889
Epoch 9/10	375/375	2s	5ms/step	0.9935	0.0230	0.9750	0.0918
Epoch 10/10	375/375	2s	6ms/step	0.9947	0.0193	0.9747	0.0926

Gambar 1.11

Model dilatih selama 10 epoch dengan batch size 128. Sebanyak 20% data training digunakan sebagai data validasi untuk memantau performa model selama pelatihan.

```
test_loss, test_accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)

313/313 ————— 1s 2ms/step - accuracy: 0.9697 - loss: 0.1021
Test Loss: 0.09088686853647232
Test Accuracy: 0.9743000268936157
```

Gambar 1.12

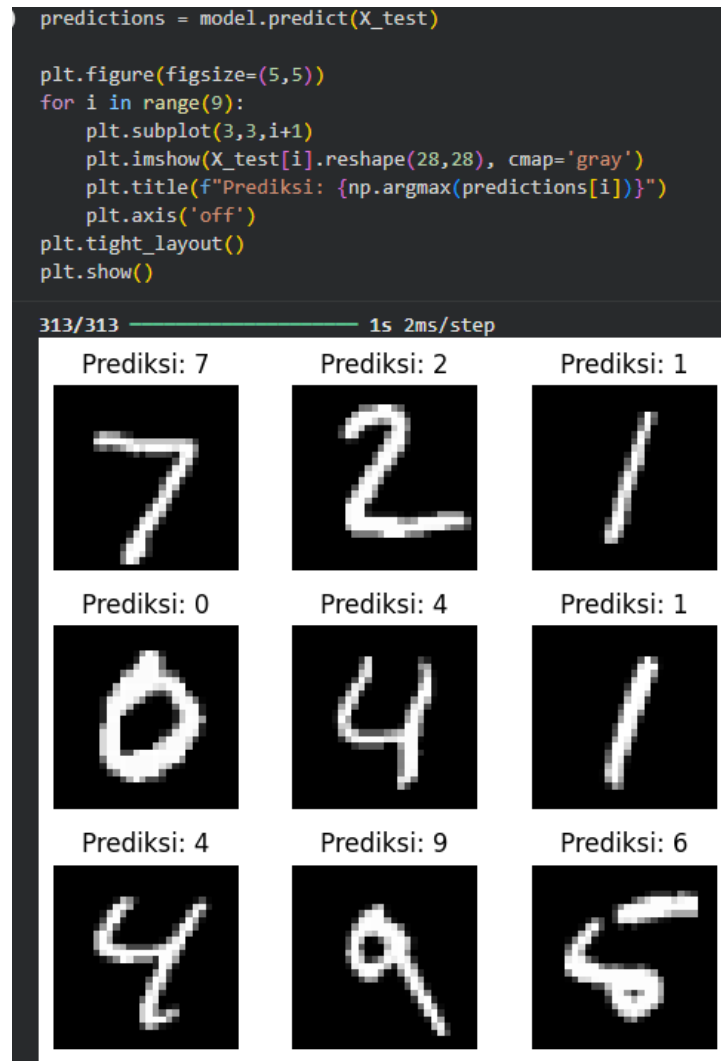
Evaluasi dilakukan menggunakan data testing untuk mengukur kemampuan generalisasi model terhadap data yang belum pernah dilihat.



Gambar 1.13

Grafik ini digunakan untuk menganalisis:

- Kestabilan pelatihan
- Indikasi overfitting atau underfitting



Gambar 1.14

menampilkan hasil prediksi model terhadap data testing untuk memverifikasi hasil klasifikasi secara visual.

Link github praktikum:

https://github.com/MuhZaidanRamdhan/machine-learning/blob/main/pertemuan13/praktikum_13/notebooks/praktikum_13.ipynb

Link github tugas:

https://github.com/MuhZaidanRamdhan/machine-learning/blob/main/pertemuan13/praktikum_mandiri/notebooks/praktikum_mandiri.ipynb